---

title: "VitalChronicle User Manual" author: "Sebastiano Romi" date: "Updated for VitalChronicle 1.0.6" lang: en-US geometry: margin=2.2cm colorlinks: true linkcolor: blue urlcolor: blue toc: true toc-depth: 3

About this manual

This is the authoritative user manual for **VitalChronicle 1.0.6**. VitalChronicle is a local-first desktop dashboard for personal Google Health data and optional local AI analysis through Ollama.

VitalChronicle is independent from Google and is not a medical device. Charts, statistical bands, anomaly indicators, correlations, and language-model output are exploratory. They do not diagnose disease, establish causality, or replace validated measurements and professional care.

VitalChronicle is and will remain free and open source. Voluntary support through Buy Me a Coffee helps maintain the project.

Interface language

VitalChronicle reads the desktop and process locale at startup, including the Linux `LANGUAGE`, `LC_ALL`, `LC_MESSAGES`, and `LANG` settings. English and Italian are included; when no supported language is detected, the interface and local-AI answers use English.

Use **Settings → Language** to select **System default** or a supported language manually. The choice is stored for future launches; restart VitalChronicle once after changing it. Advanced users and screenshot automation can temporarily override both automatic detection and the saved choice:

```
VITALCHRONICLE_LANGUAGE=it vitalchronicle
```

VitalChronicle checks for a new public release shortly after startup without blocking the interface. The network check runs at most once per day; health data are never included in the request. To check immediately, choose **Help → Check for updates**. When a newer release is available, the dialog distinguishes maintenance, feature, and major updates and can open the official GitHub release page in the system browser. If postponed, the same release is not shown again for ten days.

Additional languages are maintained by the community on Weblate and are included in releases after automatic catalogue validation.

1. What VitalChronicle does

VitalChronicle can:

- guide the user through personal Google Cloud and OAuth configuration;
- download the categories exposed by the Google Health API;
- retain a private, incremental SQLite archive on the computer;
- update missing intervals instead of downloading the complete history repeatedly;
- refresh automatically at startup and every ten minutes while running;
- visualise each metric with an appropriate chart;
- compare current values with a seven-day personal baseline;
- export individual categories as CSV or the complete archive as ZIP/JSON;
- prepare complete, time-aware summaries for a local Ollama model;
- show the model's thinking while it runs and then display the final answer.

VitalChronicle does not upload health records to the developer or to a hosted AI provider. Google receives authenticated API requests, and local AI requests are sent to the Ollama service on `127.0.0.1`.

2. System requirements

2.1 Packaged applications

Platform	Requirement
Linux	x86-64 distribution with FUSE support for AppImage, or AppImage extract-and-run
Windows	64-bit Windows 10 or 11
macOS Apple Silicon	Recent arm64 macOS release
macOS Intel	Recent x86-64 macOS release

Internet access is required for Google authentication and synchronisation. AI analysis does not require Internet after the Ollama model has been downloaded.

2.2 Running from source

- Python 3.10 or newer;
- PySide6 and the dependencies declared in `pyproject.toml`;
- a desktop session able to run Qt applications;
- optional Ollama for local AI.

2.3 Local AI hardware profiles

VitalChronicle provides two initial profiles:

- **NVIDIA GPU · 16 GB RAM** — `qwen3.5:9b` is the balanced recommendation;

- **CPU only · 32 GB RAM** — `qwen3:30b-a3b` is the balanced recommendation.

Other installed Ollama model names can be entered manually. Larger models can be very slow or may require more memory than the computer can provide.

3. Installation

3.1 Linux AppImage

1. Download the Linux AppImage from the latest release.
2. Make it executable:

   ```
   chmod +x VitalChronicle-1.0.6-linux-x86_64.AppImage
   ```

3. Start it:

   ```
   ./VitalChronicle-1.0.6-linux-x86_64.AppImage
   ```

If FUSE is unavailable, run it with:

```
APPIMAGE_EXTRACT_AND_RUN=1 ./VitalChronicle-1.0.6-linux-x86_64.AppImage
```

The AppImage is accompanied by a Sigstore bundle and release checksums.

3.2 Windows

1. Download `VitalChronicle-1.0.6-windows-x86_64.exe`.
2. Verify its SHA-256 checksum against `SHA256SUMS.txt`.
3. Start the executable.

Windows SmartScreen may warn because the file is not signed with a paid Microsoft-trusted certificate. Downloading from the official repository and matching the checksum confirms that the file is the one produced by the public workflow.

3.3 macOS

Choose the correct DMG:

- `arm64` for Apple Silicon;
- `x86_64` for Intel Macs.

Open the DMG and launch VitalChronicle. The application is ad-hoc signed but not notarised through the paid Apple Developer programme, so Gatekeeper may require explicit approval from **System Settings → Privacy & Security**.

3.4 Fedora source installation

```
sudo dnf install python3 python3-pip
git clone https://github.com/SebRoLENS/google-health-dashboard-ai.git
cd google-health-dashboard-ai
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -e .
vitalchronicle
```

3.5 Windows source installation

```
py -3 -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -e .
vitalchronicle
```

3.6 macOS source installation

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -e .
vitalchronicle
```

4. Google Cloud and OAuth setup

Each user creates a personal OAuth client. The repository deliberately does not contain a shared client secret. This keeps control of the credentials with the person whose health data are being accessed.

4.1 Create a project

1. Open Google Cloud Console.
2. Create a new project or select an existing personal project.
3. Search for **Google Health API** in the API Library and enable it.
4. Open **Google Auth Platform** for that project.

4.2 Configure Branding and Audience

Complete the required Branding fields. Under **Audience**, a personal project can remain in **Testing** mode. Add every Google account that will use VitalChronicle under **Test users**.

Testing is not mandatory. It is normally the simplest personal-use configuration and does not require public verification. For an external OAuth project in Testing, Google normally issues refresh tokens that expire after seven days when broader scopes are used. VitalChronicle will then ask the user to authenticate again.

Moving to Production may remove that testing-token limitation, but an application using sensitive or restricted scopes for a wider audience can require Google verification, published policies, and other compliance work. Personal use by a small known group can remain unverified, subject to Google's user limits and warnings.

4.3 Add Data Access scopes

Open **Data Access → Add or remove scopes**. The wizard can copy the complete list. The current read-only groups are:

```
https://www.googleapis.com/auth/googlehealth.activity_and_fitness.readonly
https://www.googleapis.com/auth/googlehealth.health_metrics_and_measurements.readonly
https://www.googleapis.com/auth/googlehealth.sleep.readonly
https://www.googleapis.com/auth/googlehealth.nutrition.readonly
https://www.googleapis.com/auth/googlehealth.ecg.readonly
https://www.googleapis.com/auth/googlehealth.irn.readonly
https://www.googleapis.com/auth/googlehealth.profile.readonly
https://www.googleapis.com/auth/googlehealth.settings.readonly
https://www.googleapis.com/auth/googlehealth.location.readonly
```

You can authorise fewer groups if you do not want the application to request every category. A denied scope simply makes the corresponding categories unavailable.

4.4 Create the OAuth client

1. Open **Clients → Create client**.
2. Select **Web application**.
3. Give the client a recognisable name, for example `VitalChronicle personal desktop`.
4. Under **Authorised redirect URIs**, add exactly:

   ```
   http://localhost:8765/
   ```

5. Create the client and immediately download its JSON file.

Google may only show the complete client secret at creation time. Keep the JSON private. Never commit it to Git, attach it to an issue, or send it to the developer.

4.5 Import and authenticate

1. Start VitalChronicle.
2. Select **Configurazione Google**.
3. Use the wizard buttons to open the relevant Google pages and copy the redirect URI or scopes.
4. Select the downloaded JSON file.
5. Confirm the configured audience and scopes.
6. Complete consent in the browser.
7. Return to VitalChronicle after the local completion page appears.

The browser redirects only to the loopback server on the same computer. Port `8765` must be free during sign-in.

5. Synchronisation and local storage

5.1 First download

Choose a period in the top bar and select **Scarica / aggiorna**. The first download can take time because many data categories are paginated independently.

One failed or unavailable category does not stop the others. VitalChronicle completes the remaining downloads and reports warnings at the end.

5.2 Incremental updates

The database stores completed date ranges for each category. Later synchronisations ask Google only for uncovered intervals. The current day remains refreshable so evolving totals can be replaced without duplicating daily roll-ups.

When credentials are available, VitalChronicle attempts a silent incremental refresh:

- shortly after startup;
- every ten minutes while the application remains open.

5.3 Food catalogues

Public food and measurement-unit catalogues are not part of automatic synchronisation. The personal nutrition log remains supported. This prevents large catalogue downloads from blocking health-data updates.

5.4 Storage locations

VitalChronicle keeps the historical internal `GoogleHealthViewer` directory identifier so existing 0.2.12 archives survive the 1.0.6 rebrand. Typical locations are:

- Linux data: `~/.local/share/GoogleHealthViewer/`;
- Linux configuration: `~/.config/GoogleHealthViewer/`;

- macOS: under `~/Library/Application Support/GoogleHealthViewer/`;
- Windows: under the current user's platform application-data directory.

The database file is named `health_data.sqlite3`. Credentials use the system keyring when possible; a permission-restricted local fallback is used when no supported keyring exists.

6. Overview

The **Panoramica** page compares the current reference day with up to seven preceding days. Missing days are not silently converted to zero; the card reports the number of days that actually contributed to a baseline.

6.1 Completion metrics

Steps, sleep duration, and active-zone minutes use progress bars because a cumulative completion metaphor is meaningful for these quantities. A card can exceed 100% when the current value surpasses the personal average.

6.2 Physiological measurements

Heart rate, HRV, oxygen saturation, respiratory rate, sleep-temperature variation, and weight are not goals to "complete". Their cards show neutral above/below-baseline differences.

The heart-rate mini-chart shows only measurements from the current calendar day. A robust curve uses five-minute medians followed by approximately fifteen-minute smoothing. The seven preceding days provide only the mean and one-standard-deviation band; they are not drawn as overlapping daily traces.

HRV, daily oxygen saturation, respiratory rate, and sleep-temperature variation display the most recent seven available daily values together with their mean and standard- deviation band. Weight always displays the most recent measurement, even when it is not from today, and is formatted to one decimal place.

7. Data explorer

Select **Esplora dati** and choose a category from the left tree. Empty categories are hidden by default and can be displayed with the checkbox.

VitalChronicle chooses a visual encoding according to the data:

- daily bars for steps, energy, workouts, and sedentary duration;
- scatter points and a local trend for dense measurements;
- lines for daily physiological summaries;
- stacked bars for sleep stages, activity intensity, and heart-rate-zone data;
- threshold bands for daily heart-rate zones.

The default vertical scale is calculated robustly from the visible time window so a few extreme values do not flatten the rest of the graph. **Tutti i valori Y** includes every visible extreme. Mouse-wheel zoom and panning operate on the time axis.

The table contains up to 5,000 visible rows and the interactive plot up to 20,000 records. Exports are not restricted by those display limits.

8. Exporting data

8.1 Current category

Select a category and choose **Esporta CSV**. The CSV contains flattened original fields and is suitable for spreadsheets or independent analysis.

8.2 Complete archive

Choose **Esporta archivio** to create a ZIP containing JSON Lines files for downloaded categories plus account/device resources. Treat exported archives as sensitive health records and store them appropriately.

9. Local AI with Ollama

9.1 Installation checks

Install Ollama from its official package, ensure the service is running, and pull a model:

```
ollama --version
curl -sS http://127.0.0.1:11434/api/version
ollama pull qwen3.5:9b
ollama list
ollama run qwen3.5:9b "Reply only with OK"
```

On Fedora, the service can be inspected with:

```
systemctl status ollama --no-pager -l
journalctl -u ollama -n 100 --no-pager
```

9.2 Hardware profile and model

Choose the closest profile, then select an installed model. VitalChronicle checks the local Ollama endpoint and periodically compares the installed model digest with the registry. Only model metadata are used for that update check; health data are not sent.

9.3 RAM and token recommendation

Open **Impostazioni AI**, enter installed RAM in GB, and request a recommendation. The estimate considers model size, CPU/GPU profile, and the context length reported by Ollama. The token value remains manually editable and is capped only when the model declares a physical context limit.

9.4 Complete analysis versus questions

Analizza tutta la cronologia ignores the period displayed for questions and prepares a summary from all locally stored categories, including secondary numeric fields, sleep stages, workout types, and structured categorical details.

Rispondi alla domanda uses the explicit period selector inside the AI page: Today, last seven days, last month, last year, all data, or a custom interval.

For totals that accumulate during the day, VitalChronicle compares the partial value with previous days at the same local time when possible. It does not treat an absent current-day value as zero or extrapolate a morning total into an entire day.

9.5 Thinking display

During a request, one output area streams the model's thinking. When a final answer is available, the same area is replaced with the answer. If a thinking-capable model ends without a final answer, VitalChronicle automatically retries with thinking disabled.

9.6 Interpretation limits

Local language models can misunderstand data, omit important context, or produce incorrect statements. Wearable measurements also contain artefacts. Review the underlying charts and raw records. Do not use AI output to change treatment, delay care, or diagnose a condition.

10. Troubleshooting

10.1 `redirect_uri_mismatch`

Check that the OAuth Web client contains exactly `http://localhost:8765/`, including the trailing slash. Google changes can take several minutes to propagate.

10.2 Access denied or unverified-app warning

Confirm that your Google account is listed under **Audience → Test users** and that the requested read-only scopes are declared under **Data Access**. An

unverified-app warning is expected for some personal projects requesting sensitive scopes.

10.3 Authentication works and later expires

An external OAuth project in Testing normally receives a seven-day refresh token for broad scopes. Authenticate again or evaluate whether Production status is appropriate for your own project.

10.4 Port 8765 is busy

Close the other process using the port and retry. On Linux:

```
ss -ltnp | grep ':8765'
```

10.5 One category fails during download

Read the warning shown at the end. VitalChronicle intentionally continues with other categories. Retry later; completed date ranges are retained.

10.6 Ollama returns HTTP 500

Test the selected model directly. If the service log reports `llama-server binary not found`, reinstall Ollama from a complete official package; downloading model weights alone does not repair a missing runtime binary.

10.7 The NVIDIA GPU is not used

Run `nvidia-smi` while the model is active and inspect the Ollama service log. Driver, container, and service environment configuration are outside VitalChronicle itself.

10.8 A chart appears empty

Confirm that the category contains meaningful values in the selected period. Some Google daily summaries arrive only after sleep processing or later in the day. Categories with no actual swimming lengths remain hidden even if an empty API record exists.

11. Removing local data

Use **Privacy → Elimina dati locali e accesso...**. This operation requires confirmation and removes the local database and stored Google credentials. Export anything you want to keep before confirming.

You can also revoke the OAuth grant from your Google Account security settings and delete the personal OAuth client from Google Cloud.

12. Security and privacy model

- OAuth client JSON and tokens are excluded by `.gitignore`.
- Local files use restrictive permissions where supported.
- Health data remain local unless the user exports them.
- Ollama analysis targets the loopback interface.
- Release workflows publish SHA-256 checksums.
- The Linux AppImage is attested using GitHub's open Sigstore infrastructure.
- Windows and macOS packages are not signed with paid platform certificates.

Report security-sensitive issues according to SECURITY.md, not in a public issue containing credentials or health data.

13. Android status

VitalChronicle 1.0.6 does not ship an APK. The current application uses desktop Qt widgets, a loopback-browser OAuth callback, desktop keyrings, and desktop chart interactions. A safe Android edition requires a dedicated mobile interface, Android OAuth credentials bound to a package name and signing certificate, mobile secure storage, lifecycle handling, and a separate validation effort. A repackaged desktop binary would not meet those requirements.

14. Updates and releases

Every source change is validated on Linux, Windows, and macOS. The automatic release workflow prepares a semantic version, refreshes synthetic screenshots, builds the PDF manual, creates an immutable tag, and dispatches native packaging jobs. Releases include platform packages, Python distributions, the PDF manual, source archive, and checksums.

See releasing.md for maintainer details.

15. Support and contact

Author: **Sebastiano Romi** sebastiano.romi@gmail.com

Use the GitHub issue tracker for reproducible bug reports and feature requests. Remove personal health data, tokens, and client secrets before attaching logs or screenshots.

VitalChronicle is and will remain open source. If you value the project, a voluntary Buy Me a Coffee contribution supports continued development without placing features behind a paywall.